PHARMACON:

Project Proposal

JAANYA JAIN

BTECH IN INFORMATION TECHNOLOGY(2026-27)

VJTI

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentors, **Bianca Sawant** and **Ankita Sagar**, and all seniors and mentors for their guidance, positive responses, and incredibly helpful feedback during the planning of this proposal. Their insights really helped me narrow down my ideas, handle my GPU constraints honestly, and structure my multi-stream architecture..

I am truly grateful to **ProjectX** for giving me this amazing opportunity. Working on *Pharmacon* has allowed me to explore an exciting intersection of chemistry and computing that I wouldn't have encountered in a regular classroom setting. It has encouraged me to dive deeper into deep learning and structural biology.

Lastly, I want to thank my friends and peers for their constant motivation and support throughout this process. Through making this proposal, I have already learned a massive amount about data parsing, identifying real-world biological gaps in standard GNN architectures, structuring multi-label evaluation metrics, and model design, and I have truly enjoyed working on it.

SECTION1: PREFACE:

1.1 About me:

Hello! I am ***Jaanya Jain***, a soon to-be second year student of Information Technology at VJTI. I have a strong interest in bioinformatics- I realised this in the summer before college- when I found myself reading about the intersection of chemistry and computing, both which I find very interesting and want to learn more about. I went down that rabbit hole- not with any particular goal, just following where the interest went.

On the technical side, I have a strong grasp of ***C++*** and working knowledge in ***Python***, both of which I also use for coding and DSA. I am a beginner in machine learning, and through this project I hope to build my understanding of how models like this work in real life, and how computational restrictions like GPU constraints affect the actual working.

1.2 Motivation for this project

Medicine and poison are not opposites- in Greek, they are the same word: pharmacon. This is not a metaphor. It is a recurring failure that affects millions of people every year. Two drugs, which are each safe on their own, can become dangerous in combination when administered in the body. ***The core challenge of this project is not just predicting whether an interaction occurs, but understanding which exact structural features of the molecules cause this.***

This project clicked for me when I read the project description. I immediately started asking questions that weren't in it. *What if the parent drug structure isn't even the molecule that ends up affecting the body? What if the dangerous interaction doesn't exist in the parent drug at all , but only in what it becomes inside you? What if the route of administration of the drug changes the metabolite's pathway and which organs it involves entirely?* And surely, drug-drug interactions vary slightly person to person, depending on pH and other bodily factors like the biochemistry of the individual.

That first question has a concrete answer. When paracetamol is taken in *excess*, it breaks down in the liver into a metabolite called NAPQI. NAPQI is what attacks liver cells and *not* paracetamol itself. The dangerous molecule only exists after the drug enters the body. Current GNN-based models take parent drug structures as input. They have never seen NAPQI. The

interaction risk they are trying to predict may not even be visible at the level they are looking at.

This is *not* a small gap. If the molecule responsible for an interaction is never given to the model, no amount of architectural advancements can recover that. It has nothing to work with. The model will learn correlations in whatever data it has, but the actual mechanism will always stay hidden.

With this project I attempt to work **backwards** from that gap: not to build on top of existing tools, but to *question* what those tools have been looking at all along. What's exciting is the chance to learn exactly how these existing solutions fall short, identifying loopholes and solving a real life problem that *directly impacts* so many people every year. This is why I chose this project.

1.3 Project Overview:

There are around 14,000 approved drugs in the world. The number of possible pairs you can make from them *goes to tens of millions*. Clinical trials will never be able to test all of them - there just isn't enough time or money. So for most drug combinations, nobody actually knows what happens when they are taken together, unless it actively affects a human being. This project tries to answer that computationally.

The main challenge is *how* you represent a drug molecule to a model. A molecule has two kinds of information: the local geometry, meaning which atoms are connected to which and how, and the global structure, meaning the overall shape and identity of the molecule. Current approaches tend to handle only one of these. *Graph-based methods* are good at local geometry but miss the bigger picture. *SMILES-based transformers* read the molecule like a sequence and get the global structure, but lose the spatial detail. Both matter, and neither alone is enough.

The project is built in ***three*** parts. First, implementing a ***Graph Convolutional Network*** on drug molecules, similar to what Decagon does. Second, using *a* ***SMILES transformer*** to capture sequence-level information. Third, ***combining*** *both* to predict drug-drug interactions, where ***cross-attention*** between atoms of Drug A and Drug B gives us the interpretability layer which is a way to see which atoms are driving the predicted interaction. If time allows, we will compare this against MolTransformer.

My **addition** to this structure starts with a question the existing approach doesn't ask: *is the model even looking at the right molecule?* Most models use the parent drug- the molecule as it

exists before you take it. But often the dangerous entity is a ***metabolite***, something the drug becomes after it enters the body***. I will add known metabolite structures from DrugBank*** *as* **input**, so the model can see what current approaches have never seen. And rather than just reporting accuracy numbers, I will ***cross-reference the atoms*** **the model flags** against documented interaction mechanisms in DrugBank and check whether the model's explanation *actually matches* what is known to cause the interaction, not just whether its predictions score give a good score.

Section 2: DOMAIN THEORY

2.1 . Molecular Representations:

The first problem this project has to solve is not a machine learning problem. It is a *chemistry problem*: how do you describe a molecule to a computer in a way that *actually* captures what makes it chemically meaningful?

There are two main ways people have tried to do this, and both of them get something right and something wrong.

- **SMILES**

SMILES stands for Simplified Molecular Input Line Entry System. The idea is simple - you take a molecule and write it out as a **string of characters**, where each character represents an atom or a bond. Benzene, for example, is written as c1ccccc1. Paracetamol is CC(=O)Nc1ccc(O)cc1.

This is *useful* because it turns a molecule into something a **sequence model can read**, the same way a transformer reads a sentence, it can read a SMILES string. The model learns relationships between parts of the molecule the way a language model learns relationships between words.

But there is a problem. **SMILES is a notation system, not a geometry system**. The string CC(=O)Nc1ccc(O)cc1 tells you *what* atoms are present and roughly how they connect, but it *does not tell you anything about the three-dimensional shape* of the molecule, the bond angles, the electron density around specific atoms. Two molecules can have the same SMILES string and behave very differently in the body depending on their spatial configuration. The sequence thus **captures global chemical identity** but **loses local atomic geometry.**

- **Molecular Graphs**

The other approach is to represent the molecule as a graph. Each atom becomes a node. Each bond becomes an edge. Node features capture the properties of the *atom* such as its element, its charge, how many hydrogens are attached to it, while the edge features capture properties of the *bond*- whether it is single, double, or aromatic.

This representation feels more natural for chemistry rather than strings- since the molecules are graphs, and this graph structure directly encodes the local geometry of the molecule - showing which atom is bonded to which, and with what kind of bond. Thus, when we run a GNN on this graph, the information will flow between atoms that are actually connected, just like real life chemical interactions.

But, there's a *catch*: A graph tells you about the local neighbourhoods, that is, what is immediately around each atom. It does not naturally capture the global identity of the molecule. Thus, we will find that two very different molecules can have similar local graphs around certain atoms, and a GNN looking only at the local structure might treat them as more similar than they actually are.

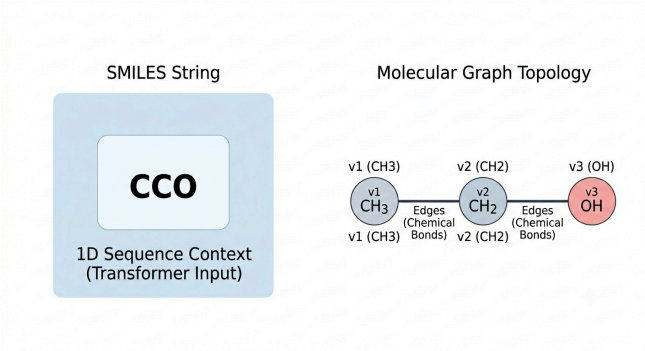

Figure 1: Dual representation breakdown mapping 1D SMILES context alongside a 2D molecular graph layout.

- **So why use both?**

This is exactly why the project uses both representations, building the bridge between them. SMILES gives the model *global* chemical context. The molecular graph gives the model *local* geometric structure. Since neither alone are sufficient, the gap between them is where the meaningful information about drug-drug interactions lives.

Another way we could *reframe* this hybrid approach is:

SMILES tells the model what *kind of molecule this is*. The graph tells the model where each atom sits and what it is doing. Thus, we need both to reason about what happens when two molecules meet.

This is also connected to a **question** I kept returning to when reading the reference papers: *why do current models that use only parent drug graphs miss certain interactions entirely?* The answer, which I will also talk about in the Gap Argument section, is partly about representation, but also about *which* molecule is being represented in the first place.

2b. Graph Convolutional Networks

A Graph Convolutional Network (GCN) is a neural network that operates *directly on a graph*. The core idea is : to learn a representation for each node, you get its information from its neighbours. We do this repeatedly across multiple layers, and each node eventually contains information about its wider neighbourhood in the graph.

For a molecule, this means each atom learns a representation that captures not just its own properties but also what is around it, like its bonded neighbours, their neighbours, and so on. After enough layers, *the atom representation encodes its local chemical environment*.

- **The propagation rule**

The mathematical operation that forms the basis of a GCN is called the **propagation rule**.

For each layer, this rule is:

$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)})$

Where,

$H^{(l)}$ is the matrix of node features at layer l. At layer zero, these are just the basic atom features like element type, charge, number of hydrogens, etc.. At each subsequent layer, it becomes a learned representation.

$\tilde{A}$ is the adjacency matrix of the graph with self-loops added. The adjacency matrix tells us which nodes are connected , so 1 if two atoms are bonded, 0 if they are not. Adding self-loops means each atom also passes its own information to itself, so it doesn't lose its own features while aggregating from neighbours.

$\tilde{D}$ is the degree matrix- which is a diagonal matrix where each entry is the number of connections a node has. The $\tilde{D}^{-1/2}$ on both sides is a normalisation step. Without it, atoms with

many neighbours would dominate the aggregation just because they have more connections, not because their information is more important. The normalisation keeps things balanced.

$\mathbf{W}^{(l)}$ is the learnable weight matrix for layer l. This is what the network trains- it learns how to transform and combine the aggregated features.

σ is the activation function, it introduces non-linearity so the network can learn complex patterns.

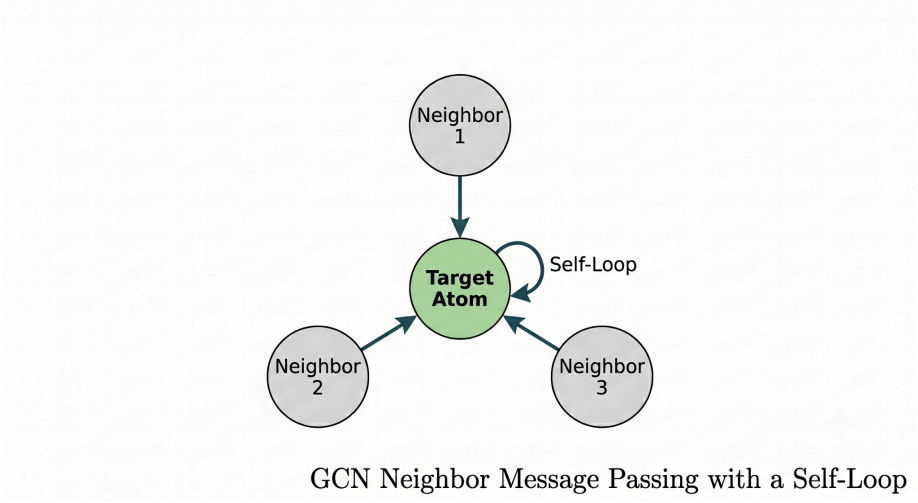

Figure 2: Graph Convolutional Network message-passing operation at the atomic node level.

So , essentially, at each layer, every atom collects feature information from its bonded neighbours, normalises it so no single neighbour dominates, transforms it through a learned weight matrix, and passes it through a non-linearity. One layer equals one hop, i.e. one step away in the graph. Two layers means information has travelled two bonds away.

- **The limitation Kipf and Welling also admit in their paper:**

This is where it got interesting. This propagation rule ***treats every neighbour equally***. When an atom aggregates information from its bonded neighbours, each neighbour contributes the same weight. There is no mechanism to say that this neighbour matters more than that one.

In chemistry, this assumption is almost never true. A hydroxyl group attached to an aromatic ring will behave very differently from a hydroxyl group attached to an aliphatic chain. The local environment changes everything. But the basic GCN has no way to distinguish that, it just averages across all neighbours uniformly.

This is exactly the problem that ***cross-attention is used to fix***, which is why the combination of GCN and attention is not just an architectural choice but it is a response to a specific *known* limitation. I will talk about this more in 5c.

- **Why use GCN over other approaches?**

But why do we use a GNN at all? There's another class of models in pharmacology called physiologically based pharmacokinetic models, or **PBPK** models, that already account for how drugs move through the body, which organs they affect, and how they are metabolised. They *already* know metabolites matter. So why not use those?

The answer is that PBPK models are *hand-engineered*. Every parameter : absorption rate, metabolic pathway, organ distribution, has to be specified *manually* for each drug. Thus scaling to tens of millions of drug pairs is not possible. And more importantly, for this project, they **output a number**. They cannot tell you *which* atoms are responsible for an interaction. A GNN gives you *scalability, learned representations, and the possibility of atom-level attribution through cross-attention*. PBPK gives you biological accuracy but at the cost of everything else.

The precise gap this project sits in is that pharmacokinetic models know metabolites matter but do not use learned representations. GNN DDI models use learned representations but ignore metabolites. This project is an attempt to sit at that intersection.

2c. Attention Mechanisms

The GCN gives each atom a proper representation of its own local environment. But once you have those representations for two drugs, you still have a problem: how do you reason about what happens when Drug A meets Drug B?

The naive approach is to just concatenate the two molecule-level representations and pass them through a classifier. But this throws away something important: ***the interaction between specific atoms*** across the two molecules. Some atoms in Drug A might be highly relevant to some atoms in Drug B, and completely irrelevant to others. A simple concatenation cannot capture that.

This is where attention comes in.

- **The attention equation:**

The core attention mechanism, introduced by Vaswani et al. in the paper Attention is All You Need, is:

Attention(Q, K, V) = softmax(QK^T / √d$_k$) V

Where,

Q is the query matrix. K is the key matrix. V is the value matrix. Intuitively, this is about information retrieval- if we search for something (the query), then we compare it against a set of possible matches (the keys). The ones that match well get high scores. Those scores then determine how much of each value we retrieve.

In practice: Q, K, and V are all *learned linear projections* of the input features. The dot product QK^T will **compute a similarity score** between every query and every key. Dividing by √d$_k$ — where d$_k$ is the dimension of the key vectors- is a ***scaling*** step we use to prevent the dot products from getting so large that the softmax saturates and gradients become tiny. The softmax turns the scores into a **probability distribution**, that is : weights that sum to one. Finally, multiplying by V produces a **weighted combination** of the values, where the weights come from how relevant each key was to each query.

- **Self-attention vs cross-attention**

There are two ways to apply this mechanism, and both matter for this project.

Self-attention is when Q, K, and V all come from the same molecule. Every atom in Drug A attends to every other atom in Drug A. This is how the SMILES transformer works, it reads the molecule as a sequence and uses self-attention to learn which parts of the molecule are relevant to each other. This is what gives the transformer its ability to capture all of the long-range dependencies that the GCN misses.

Cross-attention is different. Here, Q comes from Drug A and K, V comes from Drug B. Every atom in Drug A is asking: which atoms in Drug B are most relevant to me? The result is an attention map that shows, for each atom in Drug A, how much it attends to each atom in Drug B.

This is the interpretability layer. When the model predicts a dangerous interaction, you can look at the **cross-attention weights** and see which atom pairs across the two drugs are driving the prediction. That is the mechanism, or at least, a learned approximation of it. :/

- **The limitation Vaswani et al. acknowledge in their paper:**

Something important I found that the original paper talks about: **attention weights show *correlation, not causation***. This means, a hydroxyl group might consistently get high attention weights not because it is chemically responsible for the interaction, but because drugs with hydroxyl groups happened to *appear frequently* in dangerous pairs in the training data. The model learns the statistical pattern, not the mechanism.

This is why just reporting AUROC is *not enough.* High accuracy does not tell you whether the model has learned *the right thing for the right reasons*. The attention weights need to be **cross-referenced** against documented interaction mechanisms - which is exactly what **DrugBank provides.** This is the approach I found to be most effective for this problem**.**

If the model flags an atom, we should be able to check whether that atom *appears in the documented mechanism for that interaction*. That is the validation step that most existing work skips entirely.

- **Why attention over simpler combination methods**

We could combine the two drug representations in simpler ways too, like element-wise multiplication, concatenation, a learned linear combination. These are all reasonable baselines but none of them produce *atom-level attribution*. They tell you whether an interaction is predicted, **not where it comes from**. For a project like this, which is explicitly about *interpretability*, attention is not just a performance choice, it seems like the only mechanism that lets the model work at the level of individual atoms.

Section 3: GAP SECTION

The three foundational papers this project builds on : Decagon, Kipf and Welling, Vaswani et al., do not mention metabolites at all. Not in limitations. Not in future work. Nowhere. It is not that they tried and fell short. That question was never asked. But a **2025 review of GNN-based DDI prediction** methods published in Current Opinion in Systems Biology makes exactly this point- identifying metabolite incorporation as one of two key directions the field needs to move toward, alongside protein data. The gap is not something I am simply inferring. It is documented.

That absence is what I talk about in the next section.

The Gap Argument- The molecule the model never sees: Every GNN-based DDI model in the current literature takes parent drug structures as input. The molecule as it exists in the pill, before it enters the body. This seems like a reasonable starting point- it is, after all, what the drug actually is.

But it is not what the drug becomes.

When paracetamol is taken in excess, it does not stay as paracetamol. Inside the liver, it is metabolised into a compound called NAPQI. NAPQI is chemically reactive in a way that paracetamol is not. It binds to liver proteins and causes cell death. The dangerous entity is not the drug you swallowed, it is something that only exists after the drug enters the body and is processed.

Now consider what happens when a GNN tries to reason about a dangerous interaction involving paracetamol. It receives the parent structure. It has never seen NAPQI. The cross-attention mechanism looks for structural links between Drug A and Drug B, but if those links only exist between their metabolites, and the metabolites are not in the input, the model is looking in the wrong place entirely. Not because the architecture is wrong. Its because the input is wrong.

This is not just a hypothetical edge case. Metabolic activation is a well-documented pathway in drug toxicity. The parent drug is often pharmacologically inert — it is the metabolite that does the work, or causes the harm. A model that only sees parent structures is, by construction, blind to this entire class of interactions.

- **So what does this mean for interpretability?**

This second gap is connected to the first, but it is still distinct. Most existing models are evaluated on AUROC — *a single number* that tells you how well the model ranks positive interactions above negative ones. A *high AUROC* means the model is making **good predictions.** It does not mean the model has learned the *right reasons* for those predictions.

Cross-attention gives you atom-level weights- which is a way to see which atoms the model is attending to when it makes a prediction. But as Vaswani et al. acknowledge, attention weights show *correlation, not causation*. A hydroxyl group might get high attention not because it is responsible for the interaction, but because drugs with hydroxyl groups *appeared frequently* in

dangerous pairs during training. **The model learns the pattern. It does not necessarily learn the mechanism.**

DrugBank *documents* the actual mechanisms - which atoms, which pathways, which structural features are known to cause specific interactions. Crossreferencing the model's attention weights against those documented mechanisms is the only way to check whether the model is explaining itself correctly, not just predicting correctly. This validation step is largely absent from the existing literature.

- **A smaller observation**

There is a third gap, less developed but worth noting. Certain functional groups like hydroxyl groups, amines, aromatic rings, have consistent chemical behaviour. This knowledge exist ,in every chemistry textbook. But GNN models learn these properties from scratch, **from data,** every time. There is an argument for building these priors into the model rather than expecting it to rediscover them. This is a direction possibly worth exploring, but further down the road, not before the other two arguments.

- **Where this project lies in my opinion:**

Pharmacokinetic models know metabolites matter but they are hand-engineered, do not scale, and cannot produce atom-level attribution. GNN DDI models use learned representations and can produce attribution but they ignore metabolites and rarely validate their explanations against known mechanisms. This project sits at the intersection of those two observations. It is not trying to solve everything. It is just trying to ask a more curious question than the current tools are asking.

Capability / Feature	Decagon (GCN)	MolTransformer	Pharmacon (Ours)
Local Bond Geometry	Yes	No	Yes
Global Sequence Context	No	Yes	Yes
Metabolite-Aware Input	No	No	Yes
Atom-Level Interpretability	No	No	Yes
Mechanism-Based Validation	No	No	Yes

Figure 3:
Conceptual comparison of features across architectural baselines.

Section 4: PROPOSED APPROACH

If we only look at a drug as it exists in a pill, the model is blind. Instead of using the usual two-stream setups (Drug A and Drug B) frequently used in papers,, the pipeline I propose relies on a **four-stream architecture**. We will map the parent drugs and their primary metabolites at the exact same time. We look at both their physical 3D shape and their chemical sequence identity, and then we let them cross-examine each other using attention.

4.1 Architecture Pipeline

We break the pipeline down into three clear phases:

1. Representation Ingestion
2. Cross-entity Attention Fusion
3. Final Downstream Classification.

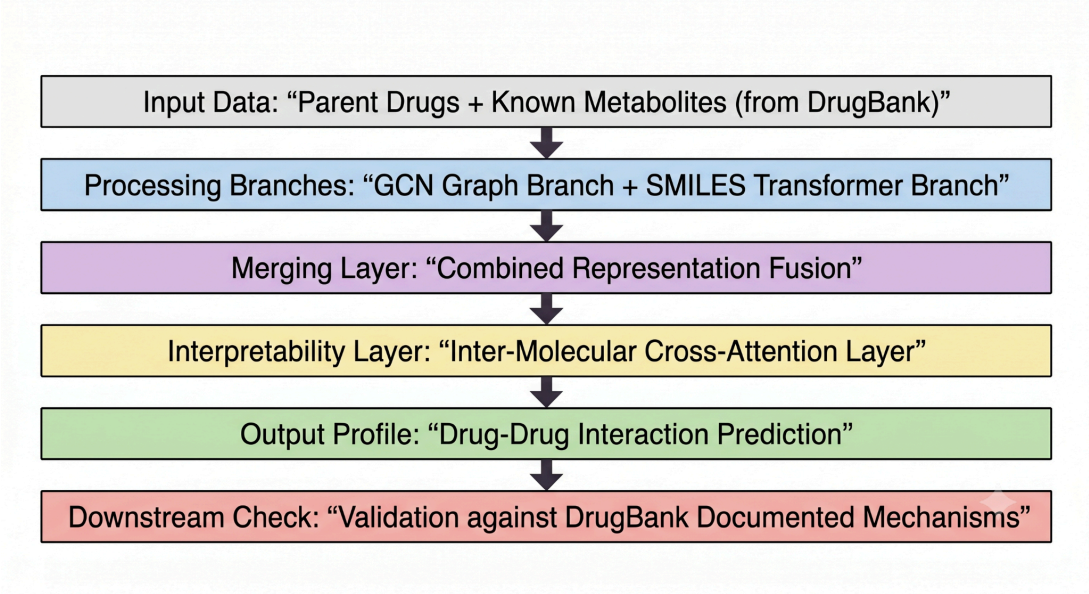

Figure 4: Visual block layout representing Pharmacon’s multi-layered four-stream workflow.

- **Phase 1: Dual-Representation Ingestion (Four Parallel Streams)**

For every single drug pair we test, we use the DrugBank Data Database to extract the parent structures and their corresponding primary metabolite.

- **Why do we stick to only one metabolite?**

Why not map the whole metabolic tree? Because the primary metabolite accounts for the vast majority of clearance in **roughly 90% of the population.** If we tried to include every single minor byproduct, our graph complexity would increase unnecessarily. Restricting it to the primary metabolite keeps the compute manageable without losing the main signal required. Thus, we evaluate four distinct items: Parent A, Parent B, Metabolite A, and Metabolite B. Each item goes through two separate encoders running side-by-side:

1. **The Spatial Stream (GCN):** We use RDKit to turn the structure into a graph G = (V, E). Nodes are individual atoms; edges are the chemical bonds. Our Graph Convolutional Network (GCN) uses standard neighbor aggregation to update atom features. This ensures the model learns the local geometry of the molecule.
2. **The Sequential Stream (SMILES Transformer):** Simultaneously, we tokenize the raw SMILES string and pass it through a native PyTorch Transformer encoder which provides us with global context..

   Once both streams finish, we take the atom **embeddings (which is the representation of data in number/vector form)** from the GCN and **concatenate** them with the token embeddings from our Transformer. This gives us four complete vectors that talk about both shape and identity: H_A, H_B, H_{MetabA}, and H_{MetabB}.

- **Phase 2: Cross-Entity Attention Fusion**

. A basic model just runs attention between H_A and H_B. But what happens if Drug A is completely harmless until its metabolite runs into the parent structure of Drug B?

To fix this, we build cross-attention layers targeting exactly four distinct cross-entity pairs:

- **Parent A -> Parent B:** Standard baseline structural interaction.
- **Parent A -> Metabolite B:** Testing how Drug A handles the broken-down version of Drug B.
- **Metabolite -> Parent B:** Tracking how a reactive byproduct (like paracetamol's NAPQI) attacks the incoming second drug.
- **Metabolite A -> Metabolite B:** Evaluating the direct interaction between the clearance products themselves.

Mathematically, our cross-attention uses one molecule's features as the Query (Q), and the interacting molecule's features as the Key (K) and Value (V):

Attention(Q, K, V) =softmax ((QK^T)/sqrt(dk)) V

This allows individual atoms inside Metabolite A to directly scan and attend to specific atoms in Parent B, flagging exactly which active centers are likely to clash.

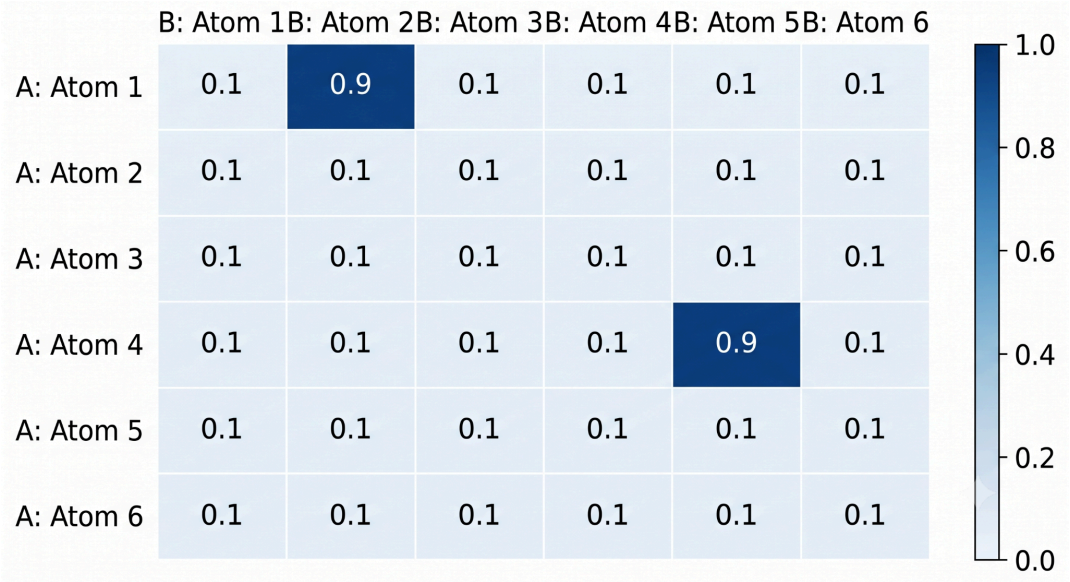

Figure 5: High-resolution cross-attention weight heatmap modeling inter-molecular atom interactions.

- **Phase 3: Classification Head**

Once the cross-attention layers finish mapping these connections, the attended vectors are brought together and combined into a final joint embedding matrix. We pass this unified matrix directly into a multi-layer perceptron (MLP) capped with a sigmoid layer. The output gives us the definitive probability for specific multi-label DDI types: whether a pair causes a drop in renal clearance or triggers acute toxicity.

4.2 Why This Approach Was chosen:

Why should we go through the massive trouble of constructing a four-stream system? Why not just put all the parent and metabolite atoms into one giant molecular graph and train a standard model on it?

The answer, intuitively lie sin the actual biology. If we bind Parent A and Metabolite A together in the initial input graph, we are telling the network that they coexist physically inside the patient's stomach. But they don't. Metabolite A doesn't even exist until Parent A undergoes first-pass metabolism via enzymes in the liver.

By keeping these structures completely separate in parallel streams, we mirror the actual biological timeline. We force the cross-attention layers to find the hidden chemical links that standard models are entirely blind to.

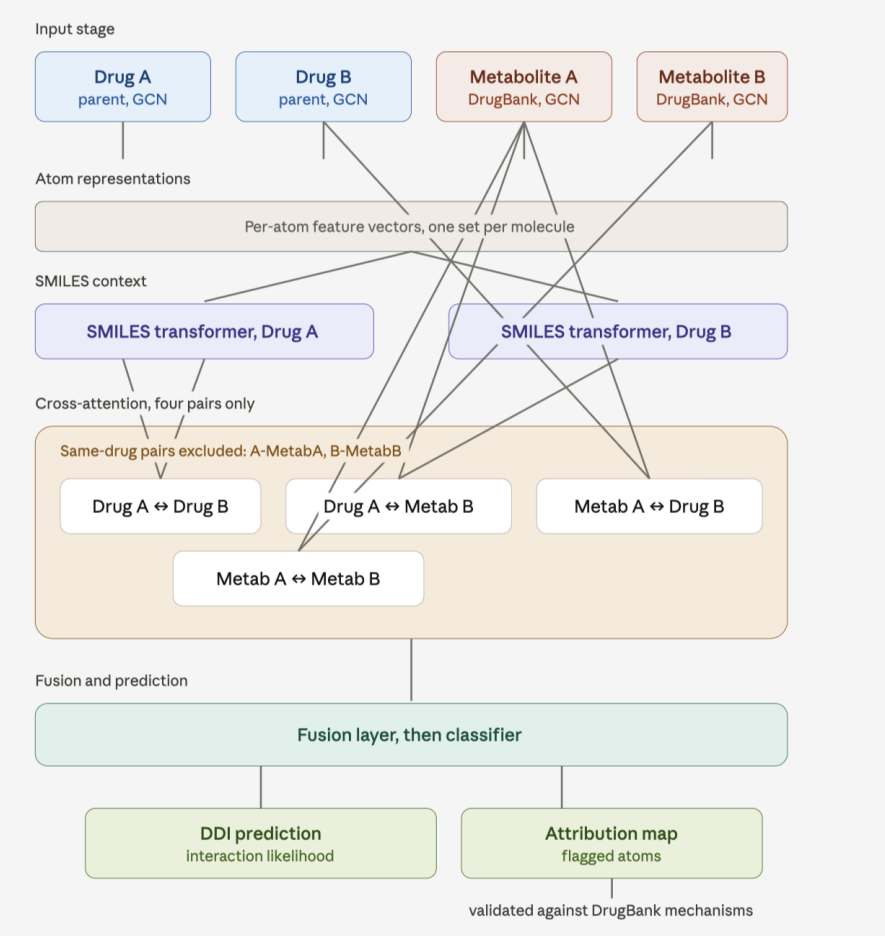

Figure 4: Detailed end-to-end multi-stream pipeline illustrating concurrent processing of parent compounds and metabolites through spatial GCN and sequential SMILES Transformer layers.

4.3 Interpretability & Validation Strategy

Another thing I notice is, the deep learning community loves to stop at a high AUROC score and call it a day. *But a high score doesn't prove your model knows an amine from an ester;* it usually just means it *got good at memorizing statistical patterns* or counting common functional groups in the training data.

Thus, we are validating our interpretability layer directly against the real chemistry documented in the **DrugBank Data Database**.

When our cross-attention layer *flags a massive weight* between suppose a hydroxyl group on Parent B and a reactive node on Metabolite A, we don't just take the network's word for it. We extract those atom indices and cross-reference them with the verified interaction sites listed in literature. If the model's highest weights land precisely on the known sites of binding or metabolic inhibition, we have a genuine validation. If they don't, we know the model is just guessing based on correlation—and we can identify andstate that transparently.

Section 5: TECH STACK AND TIMELINE

5.1. Tech Stack

My implementation strategy relies entirely on open-source libraries. The choices herey are specifically picked to handle massive datasets without blowing up our limited GPU hardware.

- **Python:** The foundational language for the entire engineering pipeline—everything from parsing raw data to staging the neural network layers.
- **PyTorch (via native nn.Transformer):** This runs our sequential text stream. Building a lean, native Transformer encoder lets us strictly restrict the token vocabulary and hidden dimensions to exactly what we need.
- **PyTorch Geometric (PyG):** The library handling our Graph Convolutional Networks (GCN). PyG implements highly optimized matrix multiplication. This matters because dense graph processing drains hardware instantly; sparse neighbor aggregation keeps the VRAM footprint low enough to train on a single mid-tier GPU.

- **RDKit:** Our main powerhouse. A neural network doesn't automatically understand what a molecule is. RDKit bridges that gap by parsing the raw chemical text strings and handles the heavy lifting of building the initial graph adjacency matrices. We also use it to extract the baseline properties for our graph nodes: things like atomic number, formal charge, and basic valence states.
- **DrugBank Data Database:** Our primary data asset.This structured dataset gives us the hard chemical pairs, the exact ground-truth classifications for multi-label drug-drug interactions, and most importantly, the text descriptions of known interaction mechanisms. We need those text files to manually audit our cross-attention outputs later on.

5.2. Timeline

Phase 1: Foundations and Data Prep (Weeks 1–4)

- **Weeks 1–2: Reading Theory and Simple Coding Tests**. These two weeks will be spent studying how message passing works in GCNs and learning how cross-attention is coded. We will write small test scripts using basic PyTorch to make sure we understand how gradients flow before trying to build the large model.
- **Weeks 3–4: Sifting and Preparing Drug Data:** Chemical data can be really messy and frustrating to parse. Most of this time will be spent writing Python scripts to sort through DrugBank data. We will use RDKit to extract four clean things for every drug: *the Parent SMILES string, the Parent Graph, the Metabolite SMILES, and the Metabolite Graph*. If a drug has broken data or errors, we will filter it out to keep our dataset clean.

Phase 2: Building the Model Architecture (Weeks 5–8)

- **Weeks 5–6: Coding the Graph and Text Encoders (GCN & Transformer)** : This is where we actually start building the AI blocks. We will divide the work to write the graph branch using PyTorch Geometric's layers and code the text sequence branch using standard PyTorch Transformer layers. By the end of Week 6, both sides must output vectors of the exact same size so they can be merged seamlessly.
- **Weeks 7–8: Combining Streams and Writing Cross-Attention:** This will be one of the trickiest parts of our coding process. We have to connect all the parallel branches together. We will write the custom cross-attention layer to map the atoms of the different molecules. To save memory and prevent crashes, we will code it to skip useless pairs (like comparing Parent A to its own Metabolite A). Finally, we will connect this to a standard Multi-Layer Perceptron (MLP) classifier to predict the interaction side effects.

Phase 3: Training & The Reality Check (Weeks 9–12)

- **Weeks 9–10: Running the Training Loops and fixing VRAM crashes:** We will set up the main training loops to run through the drug combinations. Since we face real GPU limits, a lot of our time here won't just be about checking loss curves. We will have to collaborate on optimizing memory, adjusting batch sizes, and preventing out-of-memory errors. We will track our performance using standard AUROC and AUPRC metrics
- **Weeks 11–12: Double-Checking Model Attention against Real Chemistry:** Instead of just trusting a flat accuracy percentage blindly, our team wants to verify if the model is actually learning real chemistry. In the final weeks, we will pull a few known dangerous drug pairs (like Paracetamol and its liver-attacking metabolite) and print out their attention matrices. We will use RDKit to map those numbers back to the actual atoms to verify if the model is pointing to the true, documented interaction sites.

Section 6: LIMITATIONS AND FUTURE WORK

6.1. Limitations

While adding a metabolite stream fixes a major structural blind spot in standard models, every engineering choice comes with real-world tradeoffs, and the current architecture has some clear boundaries that I would like to state honestly:

- **The Primary Metabolite Assumption:** We map exactly one primary metabolite per drug. For roughly 90% of the population, this single compound is responsible for the main clearance pathway and structural toxicity. But biology has outliers. If a patient has a specific genetic mutation that shifts their metabolism toward a rare secondary pathway, our model will completely miss it. **It is a population-level baseline, not personalized medicine.**
- **Ignoring Dose Concentration:** Our pipeline treats drug pairs as binary inputs, they are either present together or they are not. In reality, paracetamol is perfectly safe at 500 mg; it only becomes a lethal liver poison when you cross a high dosage threshold. Because our dataset lacks precise dosage dynamics, the model predicts the *possibility* of an interaction pathway, not the exact clinical threshold where it triggers. This can be

fixed in the future by adding a standard numerical feature to input the standard dosage in mg.

- **Static Chemical Environments:** We feed the model static molecular graphs and SMILES strings. But a molecule's reactivity changes depending on local pH, enzyme concentrations, and fluid dynamics. We are treating the human body like a clean, predictable beaker when it is actually a highly chaotic, shifting system.

6.2. Future Work

This proposal will act as a foundation for future work. If the core four-stream architecture proves itself, there are three logical next steps to make it clinically successful:

- **Mapping Route of Administration:** Right now, our model assumes every drug goes through standard breakdown in the liver. However, a drug given through an IV bypasses the liver entirely on its first loop, while a skin patch releases medicine at a completely different rate. In the future, we want to add a simple category tag for the delivery route so the model can pull the correct metabolite profiles automatically..
- **Integrating Shifting Organ Contexts:** The mentor validated this as well as a future milestone- To move past static molecular grids, we can include localized biological details like tissue-specific pH levels or local stomach and liver enzyme populations. This will help bridge the gap between raw chemical structures and actual human physiology.
- **Expanding the Metabolic Tree:**Sticking to just one primary metabolite per drug keeps our code simple and prevents our graphics card (GPU) from running out of memory. Once we optimize our code and reduce memory usage, we plan to expand the system to look at multi-stage breakdown charts, letting secondary and tertiary metabolites enter our cross-attention layers.
- **Deep Baseline Comparisons:** If we have extra time at the end of our development cycle, we want to run side-by-side performance tests against massive text-only models like *MolTransformer*. This will let us measure exactly how much accuracy we gain by forcing the network to look at local graph geometry.

7. References

- **Petschner, P., Nguyen, A.D., Nguyen, C.H., & Mamitsuka, H. (2025).** Machine learning for predicting drug–drug interactions: Graph neural networks and beyond. *Current Opinion in Systems Biology*, 42, 100551. https://doi.org/10.1016/j.coisb.2025.100551
- **Zitnik, M., Agrawal, M., & Leskovec, J. (2018).** Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13), i457–i466. https://doi.org/10.1093/bioinformatics/bty294
- **Kipf, T. N., & Welling, M. (2016).** Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*. https://arxiv.org/abs/1609.02907
- **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).** Attention is all you need. *Advances in Neural Information Information Processing Systems*, 30, 5998–6008. https://arxiv.org/abs/1706.03762
- **Wishart, D. S., et al. (2018).** DrugBank 5.0: a major update to the DrugBank database for 2018. *Nucleic Acids Research*, 46(D1), D1074–D1082. https://doi.org/10.1093/nar/gkx1037
- **(Dietrich, 2026, *arXiv*)**. *Pharmacogenomic Knowledge Graph Augmentation for Graph Neural Network-Based Drug-Drug Interaction Prediction.https://arxiv.org/abs/2606.07698*

PROJECT RELATED TASK ON NEXT PAGE:

PROJECT RELATED TASK:

CODE:

```python
import os
import torch
import torch.nn as nn
import torch.nn.functional as F
from rdkit import Chem
from rdkit.Chem import Draw

# Creating the folder for the result images
os.makedirs('results', exist_ok=True)

# STEP 1: we store the drug names as keys and their representations (SMILES) as the values.
inputDrugs = {
    "Paracetamol": "CC(=O)NC1=CC=C(C=C1)O",
    "NAPQI_Metabolite": "CC(=O)N=C1C=CC(=O)C=C1"
}
# header to signal start of output
print("OUTPUT")
# this will be a list that holds the parsed molecules
molecules = []
for name, smiles in inputDrugs.items():
    # when we pass the string into RDKit, it will pars eit, then will calculate valence bonds then save to variable called mol.
    mol = Chem.MolFromSmiles(smiles)
    if mol is not None:
        print(f"Successfully processed {name} structure.")
        molecules.append(mol)
        # Create and save the individual image of all ofthe input molecule
        Draw.MolToFile(mol, f"results/{name.lower()}_structure.png")
    else:
        print(f"Failed parsing {name}")

# Create a side-by-side visualization by taking both parsed molecules from out list- this shows the
# metabolic transformation of Paracetamol- when in liver- breaks down into NAPQI which is toxic to the liver cells and dangerous.
grid_img = Draw.MolsToGridImage(molecules, legends=list(inputDrugs.keys()), molsPerRow=2)
grid_img.save("results/metabolic_transformation_grid.png")
print("Saved comparison grid image to results/metabolic_transformation_grid.png\n")


# STEP 2:
class selfAttention(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.d_model = d_model
        # these create linear projection spaces in query, key and value space.
        self.q = nn.Linear(d_model, d_model)
        self.k = nn.Linear(d_model, d_model)
        self.v = nn.Linear(d_model, d_model)

    def forward(self, x):
        Q = self.q(x)
        K = self.k(x)
        V = self.v(x)
        # transposing allows us to see how much attention every character gives to every other character
        scores = torch.matmul(Q, K.transpose(-2, -1))
        d_k = x.shape[-1]
        # we scale denominator to prevent huge values of dot product which can lead to vanishing
        # gradients while training- this was mentioned in the paper

        scaled_scores = scores / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
# convert raw scores into actual probabilities that add up to one
        attention_weights = F.softmax(scaled_scores, dim=-1)
```

```
        scaled_scores = scores / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
# convert raw scores into actual probabilities that add up to one
        attention_weights = F.softmax(scaled_scores, dim=-1)
        output = torch.matmul(attention_weights, V)

        return output, attention_weights

print(" Attention Mapping- Moving to the Deep learning Model ")
# Finding/extracting the toxic metabolite smiles string
target_smiles = inputDrugs["NAPQI_Metabolite"]
# Splits that text string into characters, then counts the number of characters

tokens = list(target_smiles)
seq_len = len(tokens)

# Embedding size for vector representation is set to 16
d_model = 16
x = torch.randn(1, seq_len, d_model)

attention_model = selfAttention(d_model=d_model)
final_output, weights = attention_model(x)

print(f"Processed Sequence with : {target_smiles} ({seq_len} atomic characters)")
print(f" Shape: {weights.shape} -> Successfully parsed and visualised a drug structure along with metabolite structure.")
```

OUTPUT:

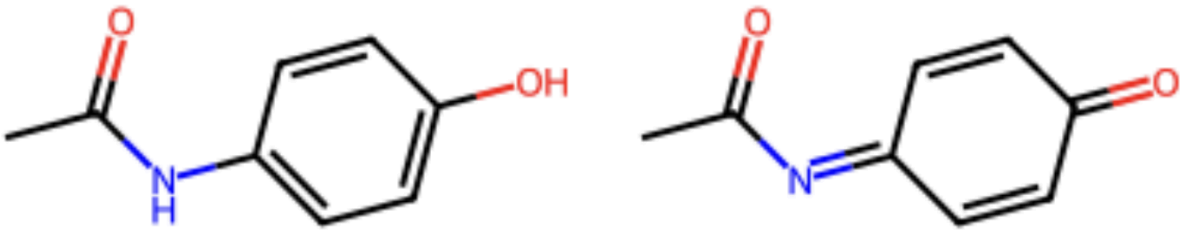

Fig 1. Side by side representation of Paracetamol (Left) and its metabolite NAPQI (Right)

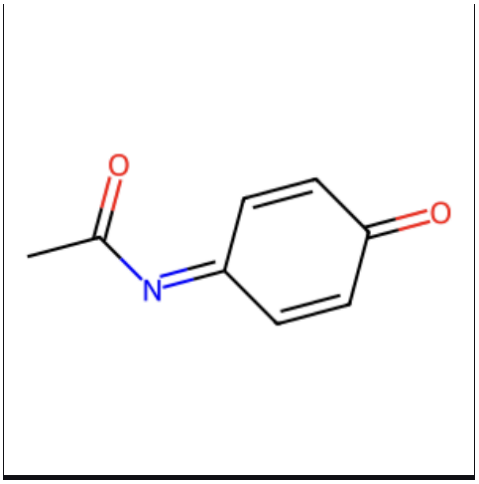

Fig 2. Representation of NAPQI (N-acetyl-p-benzoquinone imine)- METABOLITE

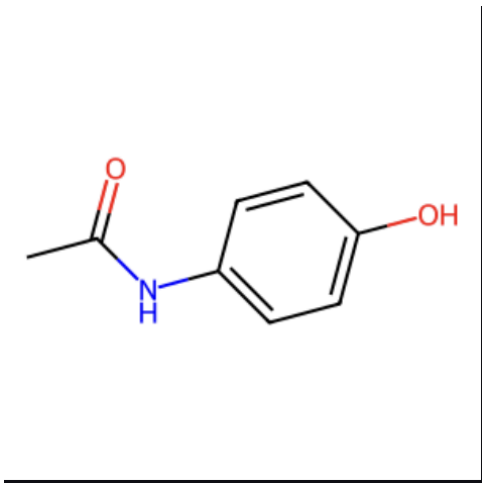

Fig 3. Representation of Paracetamol (Acetaminophen)

Terminal screenshot:

```
OUTPUT
Successfully processed Paracetamol structure.
Successfully processed NAPQI_Metabolite structure.
Saved comparison grid image to results/metabolic_transformation_grid.png

 Attention Mapping- Moving to the Deep learning Model
Processed Sequence with : CC(=O)N=C1C=CC(=O)C=C1 (22 atomic characters)
 Shape: torch.Size([1, 22, 22]) -> Successfully parsed and visualised a drug structure along with metabolite structure.
(env) jaanya@Jaanyas-MacBook-Pro-2 bioinformatics-task  %
```

THANK YOU!